- storate/production.sqlite3 is the production database. Our application would use this when we put it online; but using sqlite3 in production isn't recommended, so when we get to deployment, we'll switch this to a more robust database.

Each test method gets a freshly initialized table in the test database, loaded from the fixtures we provide. This is automatically done by the bin/rails test command but can be done separately via bin/rails db:test:prepare.

Using Fixture Data

Now that you know how to get fixture data into the database, we need to find ways of using it in our tests.

Clearly, one way would be to use the finder methods in the model to read the data. However, Rails makes it easier than that. For each fixture it loads into a test, Rails defines a method with the same name as the fixture. You can use this method to access preloaded model objects containing the fixture data: simply pass it the name of the row as defined in the YAML fixture file, and it'll return a model object containing that row's data.

In the case of our product data, calling products(:pragprog) returns a Product model containing the data we defined in the fixture. Let's use that to test the validation of unique product titles:

rails80/depot_c/test/models/product_test.rb

```
test "product is not valid without a unique title" do
  product = Product.new(title:       products(:pragprog).title,
                        description: "yyy",
                        price:       1)
  product.image.attach(io: File.open("test/fixtures/files/lorem.jpg"),
                       filename: "lorem.jpg", content_type: "image/jpeg")

  assert product.invalid?
  assert_equal [ "has already been taken" ], product.errors[:title]
end
```